Test unitaire, *mock*, *spies* et *CI*

Assurer la qualité logicielle

Aurélien Leygues (aurelien.leygues@gmail.com)

Objectifs

Connaissances à acquérir dans le cadre de ce cours :

- Savoir mettre en place un test unitaire
- Utiliser les outils *mocks* et *spies* pour correctement tester
- Automatiser l'exécution des tests avec une CI

Planning

- **Séance 1 :** introduction aux tests en dev et présentation du projet
- **Séance 2 :** premier test unitaire, *mocks*, *spies*, et mise en place de la *CI*
- **Séance 3 :** gestion BDD, tests d'intégration et scénario *testing*
- **Séance 4 :** tests E2E et notion de *code coverage*
- **Séance 5 :** plan de tests, cahier de recette et bonnes pratiques
- **Séance 6 :** approche *TDD* et développement d'une fonctionnalité
- **Séance 7 :** ouverture (*CD*, *staging*, tests A/B, etc) et préparation soutenances
- **Séance 8 :** soutenances et QCM

Test unitaire

Avec Jest

Jest

Il s'agit d'un outil de test unitaire et d'intégration pour les applications JS/TS :

- C'est un standard
- Modulable et adaptable à de nombreuse technologies
- Largement documenté

Il s'agit de l'outil que vous croiserez le plus souvent en JS/TS, il existe des équivalent dans tous les langages !

Installation

Pour installer Jest, il faut suivre la documentation dédiée à TS :

1. Ajouter **`typescript`** (si besoin)
2. Ajouter **`jest`**
3. Ajouter **`ts-jest`**
4. Générer une configuration avec **`npx ts-jest config:init`**

Documentation

```
1 {
2   "name": "testing",
3   "license": "MIT",
4   "scripts": {
5     "test": "jest"
6   },
7   "devDependencies": {
8     "@types/jest": "^30.0.0",
9     "jest": "^30.2.0",
10    "ts-jest": "^29.4.6",
11    "typescript": "^5.9.3"
12  }
13 }
```

Écrire des tests avec Jest

Jest permet d'écrire des tests de différentes manières, en voici une :

1. Créer un fichier `[nom].spec.ts`, il accueillera les tests
2. Utiliser la fonction `describe(nom_du_test, fonction_de_test)` pour regrouper des tests
3. Utiliser la fonciton `test(nom_du_test, fonction_de_test)` pour effectuer un test (synonyme de `it`)
4. Utiliser la fonction `expect(valeur)...` pour tester une valeur
5. Les fonctions `beforeEach`, `beforeAll`, `afterEach`, `afterAll` permettent de déclencher des effet de bords avant/après les tests

Le nom des fonctions de Jest n'est pas anodin et permet de construire des phrases en anglais

Premier test unitaire

Par exemple, on peut avoir une fonction qui calcule le nombre d'années pleines entre 2 dates :

```
1 // getFullYearsBetweenDates.ts
2 export function getFullYearsBetweenDates(date1: Date, date2: Date) {
3   const fromDate = date1.getTime() < date2.getTime() ? date1 : date2;
4   const toDate = fromDate === date1 ? date2 : date1;
5   const yearsDifference = toDate.getFullYear() - fromDate.getFullYear();
6   const adjustYear =
7     fromDate.getMonth() === toDate.getMonth()
8       ? fromDate.getDate() > toDate.getDate()
9       : fromDate.getMonth() > toDate.getMonth();
10   return Math.abs(yearsDifference) - Number(adjustYear);
11 }
```

Premier test unitaire

Et les tests qui vont avec, organisés dans une fonction **`describe`** (exécutés en parallèle ou séquence), puis avec des **`it`** ou **`test`** (exécutés en séquence) :

```
1  // getFullYearsBetweenDates.spec.ts
2  import { getFullYearsBetweenDates } from ".";
3
4  describe("getFullYearsBetweenDates function", () => {
5    it("should handle simples dates", () => {
6      const count = getFullYearsBetweenDates(
7        new Date(2010, 10, 1),
8        new Date(2012, 11, 1),
9      );
10     expect(count).toEqual(2);
11   });
12   it("should return complete years", () => {
13     const count = getFullYearsBetweenDates(
14       new Date(2010, 10, 1),
15       new Date(2012, 9, 1),
16     );
17     expect(count).toEqual(1);
18   });
19   it("should compare day in month if needed", () => {
20     const count = getFullYearsBetweenDates(
21       new Date(2010, 10, 10),
22       new Date(2012, 10, 1),
23     );
24     expect(count).toEqual(1);
25   });
26   test("if duration is always positive", () => {
27     const count = getFullYearsBetweenDates(
28       new Date(2012, 12, 1),
29       new Date(2010, 10, 1),
30     );
31     expect(count).toEqual(2);
32   });
33 });
```

Exécuter les tests

Jest permet d'exécuter les tests avec une commande :

- tous les fichiers `.spec` ou `.test` sont exécutés (on peut préciser le chemin d'un fichier ou dossier à exécuter si besoin)
- les tests sont exécutés qu'une fois, sauf en utilisant l'option `--watch`

```
aleygues in ~/Dev/ynov/testing/testing: yarn test
yarn run v1.22.22
$ jest
 PASS  ./getFullYearsBetweenDates.spec.ts
  getFullYearsBetweenDates function
    ✓ should handle simples dates (4 ms)
    ✓ should return complete years (1 ms)
    ✓ should compare day in month if needed
    ✓ if duration is always positive (1 ms)

Test Suites: 1 passed, 1 total
Tests:       4 passed, 4 total
Snapshots:   0 total
Time:        2.752 s
Ran all test suites.
Done in 3.83s.
```

Exercice 1 : à vous de jouer :

1. En partant de ce repo, allez dans le dossier **`server`** et installez les packages nécessaires
2. Créez un script **`test`** dans votre **`package.json`** qui exécute la commande **`jest`**
3. Vérifiez que tout est prêt en lançant **`yarn test`**, vous devriez avoir un retour de Jest indiquant qu'aucun test n'a été trouvé

Mocks

Avec Jest

Principe

Le périmètre d'un test unitaire doit être ultra-restreint et connu d'avance ! Il s'agit de tester la réponse d'une fonction pour une entrée donnée, sans aucune interaction externe

Problème : de nombreuses fonctions dépendent d'interactions externes (appels API, BDD, paquets, etc)

Ces interactions doivent donc être simulées : il s'agit du principe de bouchonnage (*mocking*, *mocks*)

Jest - *Mocks*

Tous les outils de tests unitaires supportent les *mocks*, Jest également ! La documentation se trouve ici : https://jestjs.io/docs/mock-functions

```
1 // getAverageAge.ts
2 import axios from "axios";
3
4 export async function getAverageAge() {
5   const { data } = await axios.get<{ ages: number[] }>(
6     "https://open-data.com/ages",
7   );
8   let sum = 0;
9   for (const age of data.ages) {
10    sum += age;
11  }
12  return sum / data.ages.length;
13 }
```

```
1 // getAverageAge.spec.ts
2 import axios from "axios";
3 import { getAverageAge } from "./getAverageAge";
4
5 jest.mock("axios");
6
7 describe("getAverageAge function", () => {
8   it("should compute the avg", async () => {
9     (axios.get as jest.Mock).mockResolvedValue({
10      data: { ages: [30, 40, 50] },
11    });
12    const avg = await getAverageAge();
13    expect(avg).toEqual(40);
14  });
15 });
```

Pourquoi *mocker* ?

Si vous ne *mockez* pas les interactions externes, vous vous exposez à d'importants risques :

➔ votre test dépend du bon fonctionnement d'un autre code (= faux négatifs)

➔ votre test peut débiter des quotas, et donc bloquer (ex : récupération de coordonnées GPS) voire coûter cher (ex : envois d'email ou requêtes Google API)

Dans ce cas, vous n'êtes plus en train de tester votre fonctionnalité : **votre test n'est pas bon** !

⇒ *Mocker* vous permet de tester le bon fonctionnement d'une fonction précise pour différentes combinaisons d'entrées / interactions externes

Les *mocks* sont incontournables dans les processus de tests unitaires et d'intégration, vous devez les utiliser !

Rappel : écrivez des tests uniquement pour les cas existants et pertinents, n'essayez pas de tester TOUS les cas "au cas où" (perte de sens, temps perdu, pas de gain de qualité, etc)

Exercice 2 : à vous de jouer :

1. Créez un dossier **`server/tests`**, puis un fichier avec une fonction qui retourne tous les pays qui commencent par une chaîne de caractères en vous appuyant sur https://api.first.org/data/v1/countries?limit=1000

   ```
   async function getCountriesStartingWith(srch: string):
   Promise<string[]>
   ```

2. Écrivez au moins 1 test unitaire pour cette fonction en *mockant* l'appel à l'API (vous pouvez vérifier en coupant internet)

3. **Bonus :** écrivez d'autres tests pour cette fonction

Spies

Avec Jest

Principe

Dans plusieurs langages (dont JS/TS), il est possible de passer une fonction en paramètre d'une fonction pour qu'elle soit appelée plus tard dans le temps, c'est ce qu'on appelle l'approche *callback* (aujourd'hui remplacée par les promesses)

Quand vous testez vos fonctions, vous voulez vérifier qu'une fonction *callback* soit bien appelée : vous avez besoin d'une fonction espionne ! (*spy*, *spies*)

⇒ Jest met à disposition les fonctions espionnes (via les *mocks*)

Jest - *Spies*

Les outils des langages qui supportent la programmation asynchrone supportent les espions, dans Jest, il s'agit d'un usage des *mocks*: https://jestjs.io/docs/mock-functions

```
1 // getAverageAge.ts
2 import axios from "axios";
3
4 export function getAverageAge(
5   onSuccess: (avg: number) => void,
6   onError?: (err: any) => void,
7 ) {
8   axios
9     .get<{ ages: number[] }>("https://open-data.com/ages")
10    .then(({ data }) => {
11      let sum = 0;
12      for (const age of data.ages) {
13        sum += age;
14      }
15      onSuccess(sum / data.ages.length);
16    })
17    .catch(onError);
18 }
```

```
1 // getAverageAge.spec.ts
2 import axios from "axios";
3 import { getAverageAge } from "./getAverageAge";
4
5 jest.mock("axios");
6
7 describe("getAverageAge function", () => {
8   it("should compute the avg", async () => {
9     (axios.get as jest.Mock).mockResolvedValue({
10      data: { ages: [30, 40, 50] },
11    });
12    const successSpy = jest.fn();
13    getAverageAge(successSpy);
14
15    // we need to make jest wait for the callback to happen
16    await new Promise(process.nextTick);
17
18    expect(successSpy).toHaveBeenCalled();
19    expect(successSpy).toHaveBeenCalledWith(40);
20  });
21 });
```

Pourquoi espionner ?

Les fonctions espionnes sont parfaites pour vérifier le bon appel d'une fonction passée en paramètre (ou des promesses !)

Elles s'utilisent dans 2 cas courants :

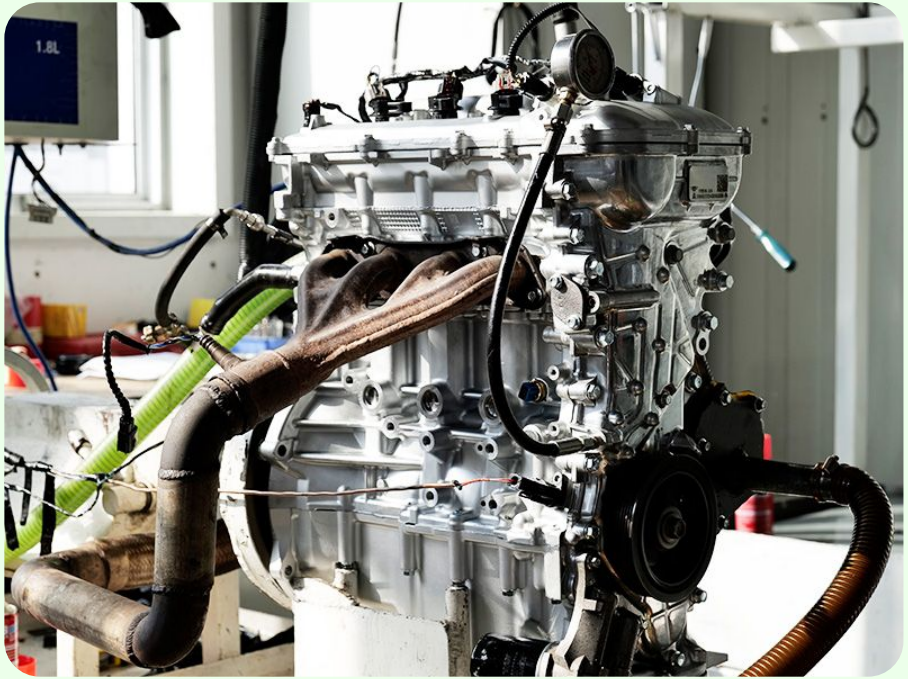
- ➔ Tester des composants *frontend* qui ont des écouteurs (**`onClick`**, **`onPress`**, etc)
- ➔ Tester des fonctions qui utilisent l'approche *callback* (**`Array.map`**, **`Array.filter`**, etc)

Mocks et *spies* peuvent tout à fait être combinés ! (et c'est le cas sur l'exemple précédent)

⇒ Quasi tous les "grands" paquets ont une documentation à propos du *testing*, pensez à aller la lire en priorité !

Par analogie, on retrouve ces principes dans le test d'un moteur (= test d'intégration) :

- consigne de vitesse = paramètre
- injection carburant = *mock*
- variation de vitesse = *spy*



Rappel : *mocker* est obligatoire pour ne pas tester en plus l'injecteur de carburant ! Les faux négatifs sont les ennemis des tests

Exercice 3 : à vous de jouer :

1. Dans le dossier **`server/tests`**, créez une fonction **`map`** qui prend en paramètre un tableau et une fonction de transformation, qui sera appliquée à chacune des entrées ; écrivez le code SANS utiliser **`Array.map`** (plus généralement, aucune méthode des tableaux)
   `function map(items: any[], changeItem: (item: any) => any): any[]`
2. Écrivez au moins 1 test unitaire qui, en donnant une fonction espionne, permet de vérifier qu'elle est bien appelée le bon nombre de fois, avec les bons paramètres
3. **Bonus :** faîtes pareil pour une fonction **`filter`** qui permet de filtrer tous les éléments d'une tableau selon le résultat (vrai/faux) de l'appel à une fonction *callback*

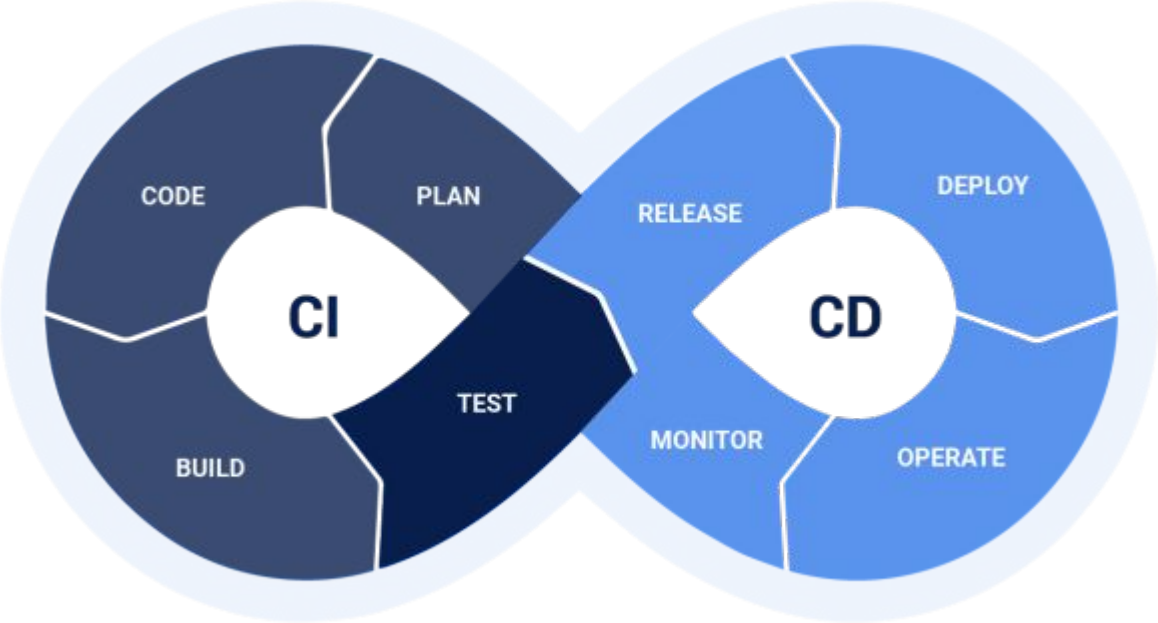
Intégration continue

Assurer automatiquement la qualité du projet

Principe

L'intégration continue (CI) s'inscrit dans les thématique DevOps et a pour objectif d'exécuter automatiquement des tâches de compilation et de test en réaction à des changements dans le code source publié (poussé sur Git)

CI & CD ⇒ approche DevOps



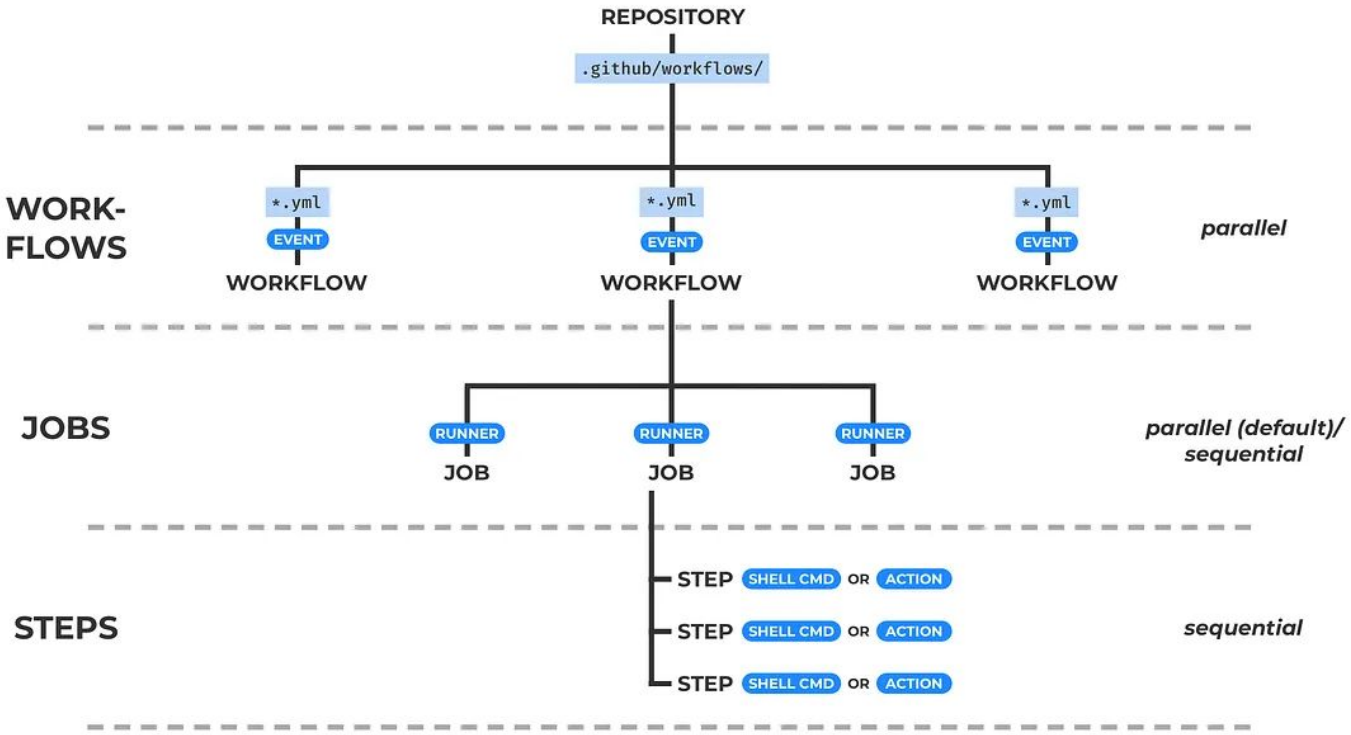
Github Actions

La CI doit être exécutée par des serveurs, Github met à disposition des serveurs pour nos CI :

- Gratuits pour les projets publics et payant (avec quelques centaines de minutes gratuites) pour les privés
- Basé sur le principe d'actions réutilisables qui s'exécutent en fonction de règles de déclenchement
- Se configure avec des fichiers YAML en respectant l'arborescences et les paramètres exigés par Github

Gitlab propose l'équivalent !

Github Actions



⇒ Démonstration Github Actions

Exercice 4 : à vous de jouer :

1. Dans le projet MFP, ajouter les outils de tests unitaires et écrivez les tests unitaires de la fonction `getDistance` déjà existante
2. Configurez ensuite les Github Actions de sorte à exécuter automatiquement les tests à chaque ajout de code (*push* sur la brancher master)

⇒ Au besoin, appuyez-vous sur la documentation des Github Actions

Pour aller plus loin :

1. Ajoutez une branche `dev` et contraignez la validation d'une PR de `dev` vers `master` à la bonne exécution des tests
2. Ajoutez `husky` (ou autre) au projet pour contraindre le passage des tests avant de pouvoir pousser le code

Bravo !

Des questions ?